

Data Structures Lab02 Description | AÖYS

- Description
- Submission view

Available from: Perşembe, 25 Eylül 2025, 1:30 PM

Due date: Cumartesi, 25 Ekim 2025, 4:30 PM

Requested files: Lab02.java (Download)

Type of work: Individual work

Interfaces

PaymentMethod

- Think of this as a **contract** for all payment types.
 - It has **one method**: `void pay(double amount)`.
 - Any payment method (Credit Card, PayPal, etc.) must implement this and provide its own behavior.
-

Payment Classes

CreditCardPayment

- Implements `PaymentMethod`.
- Has fields for `cardNumber` and `cardHolderName`.
- In `pay()`, it prints a message showing a successful payment with a masked card number.
- **Instruction:** Fill in helper methods (`verifySecurePayment`, `confirmTransaction`) with some fake or simple logic.

PayPalPayment

- Implements `PaymentMethod`.
 - Has a field for the PayPal `email`.
 - In `pay()`, it prints a message showing a successful PayPal payment.
 - **Instruction:** Fill in helper methods (`connectServer`, `retrieveInfo`) with simple mock steps.
-

Transaction Class

Transaction

- Represents a **record of money movement** (deposit, withdrawal, etc.).
 - Stores:
 - `transactionDate` (auto-set to **now**)
 - `transactionType` (e.g., "DEPOSIT", "WITHDRAWAL")
 - `amount`
 - Implements `Comparable<Transaction>` → transactions can be sorted **by date** in a `TreeSet`.
 - **Instruction:** Use `toString()` to print readable transaction history.
-

Account Hierarchy

Account (abstract class)

- Blueprint for all accounts.
- Stores:
 - `accountNumber`
 - `balance`
 - a `TreeSet<Transaction>` for sorted transaction history.
- Has methods:
 - `deposit(double amount)` → increases balance and records transaction.
 - `withdraw(double amount)` → abstract, must be defined by subclasses.
 - `printTransactionHistory()` → prints transactions in order.
- **Instruction:** Subclasses must decide withdrawal rules.

CheckingAccount

- Extends `Account`.
- Adds an `overdraftLimit`.
- `withdraw(double amount)` allows going below zero, but only up to `balance + overdraftLimit`.
- **Instruction:** Test overdraft scenarios (try withdrawing more than balance but within limit).

SavingsAccount

- Extends `Account`.
- Adds an `interestRate`.

- `withdraw(double amount)` only allows withdrawals if enough balance exists.
 - `applyInterest()` calculates interest and deposits it.
 - **Instruction:** Try calling `applyInterest()` to see balance grow.
-

Customer Hierarchy

Customer (abstract class)

- Blueprint for bank customers.
- Stores:
 - `customerId`
 - `name`
 - `accounts` (an `ArrayList<Account>`)
 - `paymentMethod` (uses **composition** with `PaymentMethod`).
- Has methods:
 - `addAccount(Account account)`
 - `setPaymentMethod(PaymentMethod method)`
 - `makePayment(double amount)` → delegates to chosen payment method.
 - `getTransactionFee()` → abstract, defined by subclasses.
- **Instruction:** Explore how changing the payment method at runtime changes payment behavior (Strategy Pattern).

RegularCustomer

- Extends `Customer`.
- Returns a **higher transaction fee** (e.g., `2.5`).

PremiumCustomer

- Extends `Customer`.
 - Returns a **lower transaction fee** (e.g., `1.0`).
-

Bank Class

Bank

- Represents the **bank system**.

- Stores:
 - `customers` in an `ArrayList<Customer>`.
 - `accountsMap` in a `HashMap<String, Account>` (for **O(1)** account lookup).
- Has methods:
 - `addCustomer(Customer customer)` → adds a new customer.
 - `openAccountForCustomer(String customerId, Account account)` → links an account to a customer and registers it in the map.
 - `findAccount(String accountNumber)` → fast lookup by account number.
 - `findCustomer(String customerId)` → search by ID.
- **Instruction:** Add multiple customers and accounts, then test lookups.

Requested files

Lab02.java